

Backend Architectural Pattern Report

1. Introduction

Designing a backend system without a clear architectural pattern leads to tightly coupled components, poor maintainability, and scalability limitations. Therefore, this project adopts a structured architectural approach to ensure the system is professional, scalable, secure, and maintainable.

The backend system is responsible for:

- Managing distributed camera nodes
- Handling fire detection events
- Managing ML model versions
- Tracking model deployment status
- Managing employee roles and actions
- Logging system activities

Due to the complexity of these responsibilities, a proper architectural pattern is required.

2. Selected Architectural Pattern

Layered Architecture Pattern (Three-Tier / N-Tier Architecture)

The system will adopt the **Layered Architecture Pattern**, also known as N-Tier Architecture.

This pattern divides the system into logical layers, where each layer has a specific responsibility.

The main layers in this project are:

1. Presentation Layer (API Layer)
2. Application Layer (Business Logic Layer)

3. Data Access Layer (Repository Layer)
 4. Database Layer (ORM Models & PostgreSQL)
 5. Configuration Layer (Environment Management)
-

3. Structure of the Architecture

The architecture is organized as follows:

1 Presentation Layer (API Layer)

- Implemented using FastAPI
- Handles HTTP requests and responses
- Validates input data
- Calls application services
- Returns structured responses

This layer does not contain business logic.

2 Application Layer (Service Layer)

- Contains business rules
- Handles model update logic
- Controls fire event processing
- Manages camera status updates
- Controls employee permissions

This layer acts as the brain of the system.

3 Data Access Layer (Repository Layer)

- Communicates with the database
- Performs CRUD operations

- Uses SQLAlchemy ORM
- Isolates database logic from business logic

This ensures that database changes do not affect application logic.

4 Database Layer

- PostgreSQL (running in Docker)
 - SQLAlchemy ORM models
 - Defines relationships between entities:
 - Employee
 - Camera
 - MLModel
 - ModelDeployment
 - FireEvent
 - CameraAction
 - Building
-

5 Configuration Layer

Sensitive data such as:

- Database host
- Database port
- Username
- Password
- Secret keys

Will be stored in a .env file and loaded using configuration management.

This ensures security and flexibility.

4. Why Layered Architecture Was Chosen

The selection of Layered Architecture is based on the following reasons:

1 Separation of Concerns

Each layer has a clear responsibility.

This reduces tight coupling between system components.

For example:

- Changing database technology does not affect API logic.
 - Changing business rules does not affect database structure.
-

2 Scalability

The system is designed to support:

- Distributed camera communication
- Continuous ML model updates
- Deployment tracking
- Failure handling
- Real-time monitoring

Layered architecture allows adding new features without restructuring the entire system.

3 Maintainability

Future modifications such as:

- Adding authentication
- Implementing role-based access control
- Integrating WebSocket alerts
- Adding logging services

Can be implemented without affecting core logic.

Testability

Business logic can be tested independently from the API layer.

This is important for:

- Model update validation
- Deployment failure handling
- Fire event classification workflows

5 **Industry Standard Practice**

Layered Architecture is widely used in enterprise backend systems because it:

- Improves code organization
- Supports microservices migration in the future
- Allows team-based development

5. How This Architecture Supports the Fire Detection System

In the Fire Detection and ML Deployment System:

1. Cameras send fire detection data.
2. The API layer receives the request.
3. The service layer processes detection logic.
4. The repository layer stores the event.
5. The database layer persists the data.
6. The system logs model deployment status.
7. Employees can intervene if deployment fails.

Each step is handled by a dedicated layer, ensuring system stability and clarity.

6. Configuration and Environment Management

To ensure secure and professional design:

Sensitive data will not be hardcoded.

Instead, environment variables will be used through a .env file.

Example:

DB_HOST=localhost

DB_PORT=5432

DB_USER=postgres

DB_PASSWORD=*****

DB_NAME=fire_system

SECRET_KEY=*****

This approach ensures:

- Security of credentials
 - Flexibility across development and production environments
 - Docker compatibility
 - Professional deployment readiness
-

7. Conclusion

The backend system for the Fire Detection and Distributed ML Camera Network will adopt a **Layered Architectural Pattern** to ensure:

- Clear separation of responsibilities
- Scalability for distributed camera systems
- Maintainability for ML model lifecycle management
- Secure configuration handling
- Professional backend engineering standards

This architectural decision ensures that the system is robust, extensible, and aligned with modern backend development practices.